

Maintenance Prédictive — API REST

Détection d'anomalie sur roulements industriels (ensemble de 6 modèles non supervisés) et estimation de durée de vie résiduelle, pour 20 capteurs IFM en production réelle — Novation City.

v3.1.0

FastAPI • Uvicorn

http://localhost:8000

23

ENDPOINTS

6

MODÈLES ML

2

RÔLES D'ACCÈS

5-60

REQ/MIN

Avant de commencer

Ce que tout appelant doit savoir avant son premier appel : authentification, limites de débit et vocabulaire des erreurs.

Authentification

En-tête `X-API-Key` comparé en temps constant. Deux rôles :

Admin	<code>API_KEYS</code> — accès complet, y compris <code>/v1/pipeline/upload</code>
Opérateur	<code>API_KEYS_OPERATOR</code> — prédiction/consultation, pas le ré-entraînement

Absent → **401** • invalide → **401** • rôle insuffisant → **403** • aucune clé côté serveur → **503** (fail-closed).

Rate limiting

Fenêtre glissante en mémoire, clé `(IP, chemin[, sensor_id])` — chaque capteur a son propre quota sur `/v1/predict`.

Dépassement → **429** avec en-tête `Retry-After`. Limite selon endpoint : de **5** à **60** req/min.

Codes d'erreur

400 / 422 Requête ou validation invalide

401 / 403 Non authentifié / non autorisé

404 Capteur ou job introuvable

429 Rate limit dépassé

503 Module optionnel indisponible

Systeme · 5

GET

/

Public

Identité de l'API : version, port, et liste résumée des endpoints principaux. Utile comme point d'entrée pour un explorateur automatique.

CURL

```
curl http://localhost:8000/
```

200 Réponse

```
{
  "api": "Maintenance Prédictive – API Unifiée",
  "version": "3.1.0",
  "port": 8000,
  "docs": "http://localhost:8000/docs",
  "endpoints": { "...": "..." }
}
```

GET**/health**

Public

Vérification de l'état de service : modèles chargés, nombre de features, capteurs actuellement en mémoire. Utilisé par le HEALTHCHECK Docker.

CURL

```
curl http://localhost:8000/health
```

200 Réponse

```
{
  "status": "ok",
  "models_loaded": true,
  "models": ["if", "lof", "ocsvm", "ecod", "hbos", "copod"],
  "features_count": 31,
  "version": "3.1.0",
  "n_sensors_in_memory": 19,
  "timestamp": "2026-08-07T10:00:00"
}
```

GET**/metrics**

Public

30/min

Métriques de performance du modèle déployé, lues depuis `models/metrics_v3.csv`
: F1, AUC-ROC, précision, rappel, et poids appris du stacking par détecteur.

CURL

```
curl http://localhost:8000/metrics
```

200 Réponse

```
{
  "model_version": "V7",
  "ensemble": "IF + ECOD + HBOS + COPOD",
  "f1_score": 0.298,
  "auc_roc": 0.9475,
  "precision": 0.373,
  "recall": 0.282,
  "n_features": 31,
  "weights": { "IF": 0.31, "ECOD": 0.28, "HBOS": 0.22, "COPOD": 0.19 }
}
```

GET

/v1/model-card

Public

30/min

« Model card » — provenance des données d'entraînement, méthodologie et limites connues de chaque composant (détection d'anomalie et RUL). Pensé pour qu'un intégrateur tiers vérifie programmatiquement sur quoi repose une prédiction avant une décision critique.

⚠ Le RUL est entraîné sur des courbes de dégradation **synthétiques** (aucune panne réelle confirmée à ce jour) — champ `rul_estimation.data_provenance.caveat`.

CURL

```
curl http://localhost:8000/v1/model-card
```

200 Réponse

```
{
  "api_version": "3.1.0",
  "anomaly_detection": {
    "models": ["IsolationForest", "LOF", "OCSVM", "ECOD", "HBOS", "COPOD"],
    "fusion": "Stacking LogisticRegression",
    "data_provenance": { "real_measurements": true }
  },
  "rul_estimation": {
    "data_provenance": { "real_failure_samples": 0 }
  },
  "known_limitations": [ "... " ]
}
```

GET

/v1/system-limits

Clé API

30/min

Liste structurée (L1 à L6) des limitations techniques connues et documentées
honnêtement : alertes non persistantes par défaut, features d'accélération toujours nulles, absence de capteur de courant, pas de déploiement edge, faux niveaux URGENT corrigés partiellement, RUL heuristique non validé sur pannes réelles.

CURL

```
curl http://localhost:8000/v1/system-limits \  
-H "X-API-Key: <votre_clé>"
```

200 Réponse

```
{  
  "version": "3.1.0",  
  "limits": [  
    { "id": "L2", "titre": "4 features d'accélération toujours nulles",  
      "statut": "LIMITATION_MATERIELLE" },  
    { "id": "L6", "titre": "RUL heuristique – pas de modèle entraîné",  
      "statut": "LIMITATION_DONNEES" }  
  ]  
}
```


POST

/v1/predict

Clé API

60/min · IP+capteur

Détection d'anomalie en temps réel via un ensemble non supervisé **IF + LOF + OCSVM + ECOD + HBOS + COPOD**, fusionné par **stacking LogisticRegression**. Un filtre de **persistance temporelle** (k=3 fenêtres consécutives par défaut) confirme l'anomalie avant de la remonter, pour réduire les faux positifs isolés.

CORPS DE LA REQUÊTE

CHAMP	TYPE	
sensor_id	string	requis
motor_id	string	optionnel
history	MeasurePoint[]	requis · min. 1 mesure

CHAMPS CLÉS DE LA RÉPONSE

risk_level — FAIBLE (score<0.25) · MODÉRÉ (<0.50) · ÉLEVÉ (<0.75) · CRITIQUE (≥0.75), renforcé à FAIBLE si **health_score**≥85 sans anomalie confirmée.

votes — nombre de modèles (0–6) dont le score individuel dépasse 0.5.

individual_models / individual_scores détaillent chaque détecteur, y compris LOF/OCSVM (diagnostic uniquement).

- ⓘ Effets de bord : met à jour l'historique glissant du capteur, alimente le buffer de vibration brute pour `/v1/spectral/{id}`, déclenche une alerte externe si le risque est CRITIQUE ou ÉLEVÉ.

ERREURS

400 history vide

CURL

```
curl -X POST http://localhost:8000/v1/predict \  
-H "X-API-Key: <votre_clé>" \  
-H "Content-Type: application/json" \  
-d '{  
  "sensor_id": "8f7f2f7e",  
  "history": [  
    { "temperature": 32.5, "vibration_x": 266.0,  
      "vibration_y": 273.0, "vibration_z": 280.0 }  
  ]  
'
```

PYTHON

```
import requests

resp = requests.post(
    "http://localhost:8000/v1/predict",
    headers={"X-API-Key": "<votre_clé>"},
    json={
        "sensor_id": "8f7f2f7e",
        "history": [
            {"temperature": 32.5, "vibration_x": 266.0,
             "vibration_y": 273.0, "vibration_z": 280.0}
        ],
    },
)
print(resp.json())
```

JS

```
const resp = await fetch("http://localhost:8000/v1/predict", {
  method: "POST",
  headers: {
    "X-API-Key": "<votre_clé>",
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    sensor_id: "8f7f2f7e",
    history: [
      { temperature: 32.5, vibration_x: 266.0,
        vibration_y: 273.0, vibration_z: 280.0 }
    ]
  })
});
const data = await resp.json();
```

200 Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "prediction": "NORMAL",
  "is_anomaly": false,
  "confidence": 0.12,
  "votes": 1,
  "risk_level": "FAIBLE",
  "anomaly_score": 0.12,
  "individual_models": {
    "IF": "NORMAL", "LOF": "ANOMALY", "OCSVM": "NORMAL",
```

```
"ECOD": "NORMAL", "HBOS": "NORMAL", "COPOD": "NORMAL"  
},  
"features": { "vib_total": 391.2, "health_score": 94.3 }  
}
```

POST

/v1/predict-rul

Clé API

60/min

Estime le temps restant avant défaillance (**RUL — Remaining Useful Life**), en combinant score de dégradation instantané, tendance temporelle, historique d'anomalies récentes, et un modèle ML (GradientBoostingRegressor) qui ne peut qu'**aggraver** le niveau déterminé par l'heuristique.

⚠ Estimation empirique — aucun modèle supervisé sur pannes réelles confirmées. Ne pas utiliser comme seule base d'une décision d'arrêt machine.

CORPS DE LA REQUÊTE

CHAMP	TYPE	
sensor_id	string	requis
history	MeasurePoint[]	requis · min. 3 mesures
risk_level, votes, confidence...	—	optionnels, typiquement issus d'un appel préalable à /v1/predict

CHAMPS CLÉS DE LA RÉPONSE

alert_level — OK (>14j) · ATTENTION (7–14j) · URGENT (3–7j) · CRITIQUE (<3j).

confidence — HAUTE (≥10 mesures) · MOYENNE (≥5) · FAIBLE (<5).

Garde-fou : health_score ≥ 85 force alert_level = OK avec un RUL plancher de 336h.

ERREURS

400 Moins de 3 mesures dans history

CURL

```
curl -X POST http://localhost:8000/v1/predict-rul \  
  -H "X-API-Key: <votre_clé>" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "sensor_id": "8f7f2f7e",  
    "risk_level": "FAIBLE",  
    "history": [  
      { "temperature": 32.5, "vibration_z": 280.0 },  
      { "temperature": 33.1, "vibration_z": 285.0 },  
      { "temperature": 33.8, "vibration_z": 290.0 }  
    ]  
  }'
```

PYTHON

```
import requests

resp = requests.post(
    "http://localhost:8000/v1/predict-rul",
    headers={"X-API-Key": "<votre_clé>"},
    json={
        "sensor_id": "8f7f2f7e",
        "risk_level": "FAIBLE",
        "history": [
            {"temperature": 32.5, "vibration_z": 280.0},
            {"temperature": 33.1, "vibration_z": 285.0},
            {"temperature": 33.8, "vibration_z": 290.0},
        ],
    },
)
print(resp.json())
```

JS

```
const resp = await fetch("http://localhost:8000/v1/predict-rul", {
  method: "POST",
  headers: {
    "X-API-Key": "<votre_clé>",
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    sensor_id: "8f7f2f7e",
    risk_level: "FAIBLE",
    history: [
      { temperature: 32.5, vibration_z: 280.0 },
      { temperature: 33.1, vibration_z: 285.0 },
      { temperature: 33.8, vibration_z: 290.0 }
    ]
  })
});
```

200 Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "rul_hours": 456.2,
  "rul_days": 19.01,
  "degradation_rate": 18.4,
  "health_score": 94.3,
  "confidence": "MOYENNE",
```

```
"alert_level": "OK",  
"recommendation": "Fonctionnement normal. RUL > 14 jours.",  
"trend": {  
  "temp_trend": 0.42,  
  "vib_total_trend": 3.1,  
  "rul_model": "heuristic_CDC + ML_gradient_boosting"  
}  
}
```

POST

/v1/iot-predict

Clé API

60/min

Pensé pour la **production sans accès base de données**. Le client envoie **une mesure à la fois** ; le serveur maintient une fenêtre glissante de 20 mesures par `sensor_id` et retourne **prédiction + RUL en un seul appel** (RUL à `null` tant que moins de 3 mesures accumulées).

CORPS DE LA REQUÊTE

CHAMP	TYPE	
sensor_id	string	requis
temperature	float	requis
vibration_x/y/z	float	requis
current, acc_*	float	optionnels

Réponse : union des champs de `/v1/predict` et `/v1/predict-rul`, plus `window_size`.

CURL

```
curl -X POST http://localhost:8000/v1/iot-predict \  
-H "X-API-Key: <votre_clé>" \  
-H "Content-Type: application/json" \  
-d '{  
  "sensor_id": "8f7f2f7e",  
  "temperature": 32.5,  
  "vibration_x": 266.0,  
  "vibration_y": 273.0,  
  "vibration_z": 280.0  
}
```

PYTHON

```
import requests

resp = requests.post(
    "http://localhost:8000/v1/iot-predict",
    headers={"X-API-Key": "<votre_clé>"},
    json={
        "sensor_id": "8f7f2f7e",
        "temperature": 32.5,
        "vibration_x": 266.0,
        "vibration_y": 273.0,
        "vibration_z": 280.0,
    },
)
print(resp.json())
```

200 Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "window_size": 4,
  "prediction": "NORMAL",
  "risk_level": "FAIBLE",
  "rul_hours": null,
  "alert_level": null,
  "features": { "...": "..." }
}
```

POST

/v1/spectral-analysis

Clé API

20/min

Analyse spectrale complète du signal de vibration : FFT (spectre de puissance, fréquences dominantes), analyse d'enveloppe (démodulation Hilbert), fréquences caractéristiques de défauts roulements (**BPFO**, **BPFI**, **BSF**, **FTF** — référence SKF 6205-2RS), décomposition en ondelettes (CWT Morlet).

PARAMÈTRES QUERY

NOM	TYPE
-----	------

rpm	float	défaut 1450.0
-----	-------	---------------

fs	float	fréquence d'échantillonnage, défaut 100.0 Hz
----	-------	--

Corps identique à /v1/predict — minimum **8 mesures** vibration_z.

ERREURS

400 Moins de 8 mesures vibration_z

503 signal_processing.py indisponible (scipy manquant)

CURL

```
curl -X POST "http://localhost:8000/v1/spectral-analysis?rpm=1450&fs=100" \
-H "X-API-Key: <votre_clé>" \
-H "Content-Type: application/json" \
-d '{"sensor_id":"8f7f2f7e","history":[{"vibration_z":280},{"vibration_z":284},
```

200 Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "signal_length": 8,
  "spectral_features": { "...": "..." },
  "bearing_analysis": { "bpfo_hz": 90.1, "bpfi_hz": 143.7 },
  "wavelet": { "...": "..." }
}
```

GET**/v1/spectral/{sensor_id}****Public**

30/min

Version publique et en lecture seule : utilise le buffer serveur des dernières valeurs vibration_z déjà reçues via /v1/predict pour ce capteur — pas de recalcul côté client, pensé pour le dashboard.

PARAMÈTRE

TYPE

rpm

float

défaut 1450.0

fs

float

défaut 100.0

CURL

```
curl "http://localhost:8000/v1/spectral/8f7f2f7e?rpm=1450"
```

200 Réponse

```
{
  "available": false,
  "reason": "Pas assez de mesures en buffer (3/8 min)."
}
```

GET**/v1/health-score/{sensor_id}****Public**

60/min

Score de santé (0–100) **normalisé par baseline propre au capteur** (calculée sur ses 20 premières mesures) — évite le biais d'une comparaison à un seuil global identique pour tous les moteurs.

Sans historique : `health_score: 100.0`, `n_records: 0` — pas de 404 sur cet endpoint.

CURL

```
curl http://localhost:8000/v1/health-score/8f7f2f7e
```

200 Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "health_score": 91.4,
  "baseline": 0.12,
  "score_method": "relatif_baseline",
  "anomaly_rate": 0.1,
  "n_records": 42,
  "trend": "STABLE"
}
```

GET

/v1/history/{sensor_id}

Public

60/min

Historique glissant des prédictions d'un capteur — en mémoire, réinitialisé au redémarrage (sauf restauration depuis `anomaly_history_persist.json` pour les capteurs IFM connus).

PARAMÈTRE

TYPE

limit**int**

défaut 20 · max 50 en mémoire

ERREURS

404 Aucun historique pour ce capteur

CURL

```
curl "http://localhost:8000/v1/history/8f7f2f7e?limit=20"
```

200

Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "n_total": 50,
  "avg_score": 0.14,
  "trend": "STABLE",
  "history": [
    { "timestamp": "2026-08-07T09:58:00", "score": 0.11 }
  ]
}
```

GET

/v1/alert-level/{sensor_id}

Public

60/min

Niveau d'alerte **consolidé** sur les 10 dernières prédictions, pensé pour un feu tricolore de dashboard.

OK

ATTENTION

URGENT

CRITIQUE

CURL

```
curl http://localhost:8000/v1/alert-level/8f7f2f7e
```

200 Réponse

```
{
  "sensor_id": "8f7f2f7e",
  "alert_level": "OK",
  "color": "green",
  "icon": "●",
  "avg_score": 0.12,
  "message": "Fonctionnement nominal."
}
```

GET

/v1/results

Public

60/min

Dernières prédictions (anomalie + RUL fusionnées) de tous les capteurs actifs, ou d'un capteur précis via ?sensor_id=. Mis à jour à chaque appel /v1/predict ou /v1/predict-rul.

ERREURS

404 sensor_id fourni mais introuvable

CURL

```
curl "http://localhost:8000/v1/results?sensor_id=8f7f2f7e"
```

200 Réponse

```
{
  "status": "ok",
  "n_sensors": 1,
  "results": [ { "sensor_id": "8f7f2f7e", "predict": {"...":"..."} } ]
}
```

Données · 2

GET

/sensors

Public

60/min

Liste des capteurs actifs — source prioritaire : historique temps réel en mémoire (rempli par /v1/predict), repli sur le CSV historique si aucune activité récente.

CURL

```
curl http://localhost:8000/sensors
```

200 Réponse

```
{
  "sensors": [
    { "sensor_id": "8f7f2f7e", "n_measures": 128,
      "anomaly_rate": 0.05, "avg_health": 93.2 }
  ],
  "source": "realtime"
}
```

GET

/anomalies

Public

60/min

Anomalies filtrées par score minimum, depuis le jeu de résultats historique.

PARAMÈTRE

TYPE

min_score

float

défaut 0.5

limit

int

défaut 100

CURL

```
curl "http://localhost:8000/anomalies?min_score=0.6&limit=50"
```

200

Réponse

```
{
  "n_anomalies": 3,
  "anomalies": [
    { "sensor_id": "2c6254af", "anomaly_score": 0.81, "risk_level": "CRITIQUE" }
  ]
}
```

Reporting · 1

GET

/v1/report

Clé API

20/min

Génère un rapport de maintenance à partir des données temps réel.

PARAMÈTRE

TYPE

type	string	daily (défaut) · weekly · monthly · full
format	string	json (défaut) · html
sensor_id	string	optionnel — filtre un capteur

ERREURS

400 type invalide

503 reporting_module.py indisponible

CURL

```
curl "http://localhost:8000/v1/report?type=weekly&format=json" \
-H "X-API-Key: <votre_clé>"
```

200 Réponse

```
{
  "report_type": "weekly",
  "generated_at": "2026-08-07T10:00:00",
  "kpis": { "n_sensors": 19, "n_anomalies": 4 }
}
```

Alertes · 2

GET

/v1/alerts

Public

30/min

Historique des alertes externes envoyées (email/webhook/SMS) par AlertManager, avec statut de livraison par canal — booléens uniquement, jamais d'adresse, URL ou identifiant exposé.

PARAMÈTRE

TYPE

limit

int

défaut 50

ⓘ Ne porte pas de dépendance `require_api_key` dans le code actuel — accessible sans clé, comme les autres endpoints de monitoring en lecture seule.

CURL

```
curl "http://localhost:8000/v1/alerts?limit=20"
```

200 Réponse

```
{
  "enabled": true,
  "alerts": [
    { "sensor_id": "8f7f2f7e", "risk_level": "CRITIQUE",
      "channels_sent": { "email": true, "webhook": false, "sms": false } }
  ]
}
```

GET

/v1/alerts/stats

Public

30/min

Statistiques globales du gestionnaire d'alertes : total envoyées, répartition par niveau, cooldowns actifs.

CURL

```
curl http://localhost:8000/v1/alerts/stats
```

200 Réponse

```
{
  "enabled": true,
  "total_alerts": 12,
  "by_level": { "CRITIQUE": 3, "URGENT": 9 }
}
```

Pipeline · 4

GET

/pipeline

Public

Sert la page HTML d'upload (`pipeline_upload.html`). Exclue du schéma OpenAPI (`include_in_schema=False`).

CURL

```
curl http://localhost:8000/pipeline
```

POST

/v1/pipeline/upload

Admin uniquement

5/min

Reçoit un fichier .sql (flux par blocs de 4 Mo), puis démarre en tâche de fond : parsing SQL → CSV, entraînement des 6 modèles, rechargement à chaud dans l'API — sans redémarrage.

CHAMP (MULTIPART)

TYPE

file

.sql

requis

train_mode

string

"full" (défaut) ou "fast"



Une clé **opérateur valide** reçoit **403** ici (authenticifiée mais non autorisée) — seule une clé API_KEYS (admin) passe.

ERREURS

400 Extension différente de .sql

403 Clé opérateur (non admin)

CURL

```
curl -X POST http://localhost:8000/v1/pipeline/upload \  
  -H "X-API-Key: <clé_admin>" \  
  -F "file=@dump_ai_cp.sql" \  
  -F "train_mode=full"
```

200 Réponse

```
{  
  "job_id": "a1b2c3d4e5f6",  
  "status": "running",  
  "filename": "dump_ai_cp.sql",  
  "size_mb": 87.4,  
  "poll_url": "/v1/pipeline/status/a1b2c3d4e5f6"  
}
```

GET

/v1/pipeline/status/{job_id}

Clé API

60/min

Statut d'un job en cours — à interroger toutes les 2-3 secondes.

PARAMÈTRE	TYPE
-----------	------

since	int	défaut 0 — ne retourne que les logs postérieurs à cet index
-------	-----	---

ERREURS

404 job_id inconnu

CURL

```
curl "http://localhost:8000/v1/pipeline/status/a1b2c3d4e5f6?since=12" \  
-H "X-API-Key: <votre_clé>"
```

200 Réponse

```
{  
  "job_id": "a1b2c3d4e5f6",  
  "status": "running",  
  "step": 4,  
  "progress": 58,  
  "new_logs": [ { "text": "✅ LOF entraîné", "type": "s" } ]  
}
```

GET

/v1/pipeline/jobs

Clé API

30/min

Liste des 10 derniers jobs de pipeline.

CURL

```
curl http://localhost:8000/v1/pipeline/jobs \  
-H "X-API-Key: <votre_clé>"
```

200 Réponse

```
{  
  "total": 3,  
  "jobs": [  
    { "job_id": "a1b2c3d4e5f6", "status": "done", "progress": 100 }  
  ]  
}
```

Généré à partir de `api_unified_pythagore.py`, `auth.py` et `rate_limiter.py` — v3.1.0. Documentation détaillée : `API_DOCUMENTATION.md` · rapport imprimable : `API_DOCUMENTATION_TECHNIQUE.pdf`.